



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ  
(национальный исследовательский университет)»

---

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика» Кафедра 806  
Группа М8О-409Б-22 Направление подготовки 01.03.02 «Прикладная математика и информатика»  
Профиль Информатика  
Квалификация: бакалавр

---

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА  
БАКАЛАВРА**

На тему: Система индексации ончейн-данных

---

Автор ВКРБ: Ефименко Кирилл Игоревич ( )  
Руководитель: Филимонов Николай Сергеевич ( )  
Консультант: Булакина Мария Борисовна ( )  
Консультант: ( )  
Рецензент: Филимонов Николай Сергеевич ( )

**К защите допустить**

Заведующий кафедрой № 806 Крылов Сергей Сергеевич ( )  
\_\_\_\_ мая 2026 года

## РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 54 страниц, 7 рисунков, 19 таблиц, 13 использованных источников, 3 приложений.

ОНЧЕЙН-ДАННЫЕ, ИНДЕКСАЦИЯ ДАННЫХ, БЛОКЧЕЙН BITCOIN, МОДУЛЬНЫЙ МОНОЛИТ, STATELESS-АРХИТЕКТУРА, RUST, POSTGRESQL, REST API, MEMPOOL, REORG, UTXO

Выпускная квалификационная работа посвящена разработке системы индексации ончейн-данных сети Bitcoin. Актуальность темы определяется необходимостью создания специализированного программного слоя, обеспечивающего структурированное хранение и прикладную выдачу блокчейн-данных с предсказуемой производительностью чтения, устойчивостью к изменению канонической цепи и возможностью интеграции с внешними информационными системами.

Объектом исследования является процесс получения, нормализации, хранения и предоставления ончейн-данных сети Bitcoin. Предметом исследования выступают архитектурные и алгоритмические решения, обеспечивающие индексацию блоков, транзакций, адресных балансов, UTXO и неподтвержденных транзакций, а также согласованное предоставление этих данных прикладным потребителям.

Цель работы состоит в проектировании и реализации системы индексации ончейн-данных, которая получает данные из узла Bitcoin Core по JSON-RPC, нормализует их, сохраняет в PostgreSQL и предоставляет через REST API, административный интерфейс и командные средства сопровождения. В ходе работы были проанализированы особенности предметной области, сформированы требования к системе, спроектированы архитектура и модель данных, реализованы механизмы индексации подтвержденной цепи, обработки reorg, синхронизации mempool, мониторинга и контейнерного развертывания.

Практическим результатом работы является программный комплекс Bitcoin Blockchain Indexer, включающий серверное приложение на Rust, схему и миграции базы данных, административную панель, командный интерфейс, эксплуатационную документацию и интеграционные тесты. Разработанное решение проверено на уровне ключевых интеграционных сценариев и может использоваться как основа инфраструктурного компонента

аналитических, исследовательских и сервисных систем, работающих с ончейн-данными Bitcoin.

## СОДЕРЖАНИЕ

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ . . . . .                                                    | 6  |
| ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ . . . . .                                        | 8  |
| ВВЕДЕНИЕ . . . . .                                                                 | 9  |
| 1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ                                 | 10 |
| 1.1 Анализ существующих подходов к индексации ончейн-данных                        | 10 |
| 1.2 Понятие и особенности ончейн-данных . . . . .                                  | 12 |
| 1.3 Особенности блокчейна Bitcoin как источника данных . . . . .                   | 13 |
| 1.4 Проблемы извлечения, хранения и предоставления ончейн-данных . . . . .         | 15 |
| 1.5 Анализ существующих подходов и средств индексации блокчейн-данных . . . . .    | 17 |
| 1.6 Постановка задачи разработки системы индексации ончейн-данных . . . . .        | 19 |
| 2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ ИНДЕКСАЦИИ ОНЧЕЙН-ДАнных                                  | 21 |
| 2.1 Формирование функциональных и нефункциональных требований . . . . .            | 22 |
| 2.2 Выбор архитектурного подхода и проектирование структуры модулей . . . . .      | 26 |
| 2.3 Проектирование модели данных и схемы хранения . . . . .                        | 29 |
| 2.4 Проектирование интерфейсов взаимодействия и сценариев работы системы . . . . . | 31 |
| 3 РАЗРАБОТКА СИСТЕМЫ ИНДЕКСАЦИИ ОНЧЕЙН-ДАнных                                      | 34 |
| 3.1 Реализация серверной части системы . . . . .                                   | 34 |
| 3.2 Реализация механизма индексации блоков и транзакций . . . . .                  | 35 |
| 3.3 Реализация управления заданиями индексирования, mempool и georg . . . . .      | 36 |
| 3.4 Реализация API, конфигурирования и мониторинга . . . . .                       | 38 |
| 3.5 Кодовые фрагменты, иллюстрирующие реализацию . . . . .                         | 39 |
| 4 ИССЛЕДОВАНИЕ И ОЦЕНКА КАЧЕСТВА РАЗРАБОТАННОЙ СИСТЕМЫ                             | 41 |
| 4.1 Цели и задачи испытаний . . . . .                                              | 41 |
| 4.2 Программа и методика испытаний . . . . .                                       | 41 |

|     |                                                                     |    |
|-----|---------------------------------------------------------------------|----|
| 4.3 | Проверка корректности функциональных возможностей . . . .           | 43 |
| 4.4 | Оценка устойчивости к геог и изменениям состояния mempool           | 44 |
| 4.5 | Оценка эксплуатационных характеристик и наблюдаемости . .           | 44 |
| 4.6 | Анализ результатов испытаний . . . . .                              | 45 |
| 5   | ПРАКТИЧЕСКАЯ ЗНАЧИМОСТЬ И ПЕРСПЕКТИВЫ РАЗВИТИЯ<br>СИСТЕМЫ . . . . . | 46 |
| 5.1 | Практическая значимость разработанного решения . . . . .            | 46 |
| 5.2 | Возможности применения системы в прикладных сервисах . .            | 47 |
| 5.3 | Ограничения текущей версии системы . . . . .                        | 47 |
| 5.4 | Направления дальнейшего развития системы . . . . .                  | 48 |
| 5.5 | Выводы по результатам работы . . . . .                              | 49 |
|     | ЗАКЛЮЧЕНИЕ . . . . .                                                | 50 |
|     | СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . . .                          | 51 |
|     | ПРИЛОЖЕНИЕ А Диаграмма контейнерного развёртывания . . . . .        | 52 |
|     | ПРИЛОЖЕНИЕ Б Структура репозитория проекта . . . . .                | 53 |
|     | ПРИЛОЖЕНИЕ В Фрагмент конфигурации индексатора . . . . .            | 54 |

## ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяют следующие термины с соответствующими определениями:

Ончейн-данные — данные, зафиксированные в блокчейне, а также производные представления, формируемые на их основе для прикладного анализа

Индексер — серверное приложение, которое получает данные блокчейна, нормализует их, сохраняет в базе данных и предоставляет прикладной API

Bitcoin Core — программная реализация узла сети Bitcoin, используемая как доверенный источник первичных блоков и транзакций

Bitcoin — децентрализованная блокчейн-сеть и одноименная криптовалютная система, данные которой являются предметом индексации

Каноническая цепь — актуальная ветвь блокчейна Bitcoin, признаваемая основной в рассматриваемый момент времени

Задание индексирования — управляемый процесс обработки блокчейн-данных с собственным идентификатором, статусом, прогрессом и параметрами выполнения

Reorg — смена канонической ветви блокчейна, из-за которой ранее принятые блоки и транзакции становятся неканоническими

Mempool — множество неподтвержденных транзакций, известных узлу Bitcoin Core и ожидающих включения в блок

Stateless-сервис — сервис, который не хранит критичное состояние локально между перезапусками и размещает его во внешнем хранилище

Адресный индекс — структура данных, позволяющая быстро находить транзакции, выходы, балансы и mempool-события, связанные с конкретным адресом

Адресный баланс — агрегированное значение суммы непотраченных выходов или исторического состояния адреса на определенный момент цепи

UTXO — непотраченный выход транзакции, который может быть использован как вход последующей транзакции

PostgreSQL — реляционная система управления базами данных, используемая для хранения блоков, транзакций, UTXO, балансов, заданий и метрик состояния

Rust — язык программирования, на котором реализована серверная часть

системы

Python — язык программирования, используемый для реализации командного интерфейса проекта

React — JavaScript-библиотека, используемая при реализации административной панели

Vite — инструмент сборки frontend-приложений, применяемый для административной панели

Docker — платформа контейнеризации, применяемая для воспроизводимой сборки и развертывания компонентов системы

Docker Compose — средство описания и запуска набора связанных контейнеров проекта

nginx — веб-сервер и reverse проху, используемый в проектной схеме защищенного контура и контейнерного развертывания

Prometheus — система мониторинга, для которой приложение публикует метрики состояния индексатора и его фоновых процессов

Satoshi — минимальная неделимая единица Bitcoin, используемая для представления значений выходов и балансов

OpenAPI — спецификация описания HTTP API, используемая для документирования контрактов backend-сервиса

Swagger UI — веб-интерфейс для просмотра и интерактивного вызова API на основе OpenAPI-описания

Regtest — локальный режим сети Bitcoin, предназначенный для воспроизводимых тестов с управляемой генерацией блоков

Testcontainers — инструмент запуска временных контейнеров в автоматизированных тестах, используемый для проверки сценариев с PostgreSQL

## ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяют следующие сокращения и обозначения:

API — Application Programming Interface

C4 — Context, Containers, Components, Code

CLI — Command Line Interface

БД — база данных

DFD — Data Flow Diagram

ETL — Extract, Transform, Load

HTTP — HyperText Transfer Protocol

HTTPS — HyperText Transfer Protocol Secure

JSON — JavaScript Object Notation

JSON-RPC — протокол удаленного вызова процедур поверх JSON

mTLS — mutual TLS

MVP — Minimum Viable Product

OpenAPI — OpenAPI Specification

FK — Foreign Key

PK — Primary Key

REST — Representational State Transfer

RPC — Remote Procedure Call

SQL — Structured Query Language

TLS — Transport Layer Security

UI — User Interface

UTXO — Unspent Transaction Output

YAML — YAML Ain't Markup Language



## ВВЕДЕНИЕ

Прямой доступ к блокчейн-данным через узел Bitcoin Core удобен для низкоуровневого взаимодействия, однако плохо подходит для регулярных прикладных выборок, аналитики и построения внешних программных интерфейсов. Узел ориентирован на поддержание консенсуса, хранение блокчейн-истории и обслуживание RPC-вызовов, но не предоставляет слой прикладных представлений, необходимых для внешних систем: исторических балансов адресов, наборов UTXO, фильтрации транзакций по адресам и диапазонам времени, а также механизмов контроля жизненного цикла индексирования [1; 2].

В прикладных сценариях это приводит к высокой нагрузке на RPC-интерфейс, необходимости повторного расчета производных агрегатов и риску неконсистентности данных при локальном пересмотре канонической цепи. Следовательно, возникает задача построения отдельной системы индексации блокчейн-данных, отделяющей контур получения исходной информации от контура ее структурированного хранения и предоставления прикладным клиентам.

В настоящей работе рассматривается система индексации блокчейн-данных сети Bitcoin, реализуемая в форме программного комплекса Bitcoin Blockchain Indexer. Система получает данные от доверенного узла, сохраняет их в PostgreSQL [3], предоставляет доступ к ним через REST API, поддерживает управление заданиями индексирования, мониторинг состояния узлов и обслуживание запросов к данным mempool.

Целью выпускной квалификационной работы является проектирование и реализация системы индексации блокчейн-данных, обеспечивающей согласованное хранение и прикладную выдачу данных сети Bitcoin. Для достижения поставленной цели в работе решаются задачи анализа предметной области, формирования требований, обоснования архитектурного подхода, проектирования модели данных, реализации ключевых модулей и проверки качества разработанного решения.

# **1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ**

Разработка системы индексации ончейн-данных требует рассмотрения как специфики блокчейн-сетей, так и особенностей построения прикладных информационных систем, работающих с большими потоками событий. В отличие от традиционных транзакционных систем, блокчейн Bitcoin выступает одновременно распределенным журналом, средой консенсуса и источником данных для внешних программных сервисов. Это накладывает ограничения на способы извлечения данных, их интерпретацию, хранение и последующее предоставление через прикладные интерфейсы.

Для обоснования проектных решений по теме ВКР необходимо определить сущность ончейн-данных, выявить особенности блокчейна Bitcoin как источника информации, проанализировать ограничения прямой работы прикладных систем с узлом сети, а также сопоставить существующие подходы к индексации блокчейн-данных. Результат такого анализа позволяет не только сформулировать задачу разработки системы Bitcoin Blockchain Indexer, но и доказательно обосновать необходимость выделения специализированного слоя индексации и хранения данных.

## **1.1 Анализ существующих подходов к индексации ончейн-данных**

Задача предоставления прикладного доступа к данным Bitcoin может решаться разными способами: прямым использованием RPC узла, развертыванием готового индексатора, применением аналитических платформ или выгрузкой данных в пакетном ETL-формате. Каждый подход закрывает часть требований, однако не все из них одинаково подходят для системы, в которой необходимы управляемые задания индексирования, REST API, PostgreSQL-хранилище, контроль георг, mempool и эксплуатационная наблюдаемость.

Bitcoin Core предоставляет канонический источник данных и RPC-интерфейс, но его назначение состоит прежде всего в поддержании узла сети, а не в обслуживании прикладных выборок по адресам и агрегатам [1]. Индексаторы семейства electrs ориентированы на быстрые запросы кошельков и используют собственное хранилище индексов; проект Esplora дополняет такой подход web-интерфейсом и HTTP API для обозревателя блоков [4; 5]. Аналитическая платформа BlockSci предназначена для исследовательского

анализа блокчейн-данных и использует аналитическую модель хранения, но не является транзакционным backend-сервисом для оперативной выдачи данных внешним приложениям [6]. Проект Bitcoin ETL решает задачу пакетной выгрузки блоков и транзакций, включая сценарии потоковой передачи, однако его модель ближе к подготовке датасетов, чем к сопровождению управляемого индекса с API [7]. Публичные explorer API удобны для быстрых интеграций, но не дают полного контроля над источником данных, политикой хранения, ограничениями доступа и воспроизводимостью результатов.

Таблица 1 – Сравнение подходов к получению и индексации данных Bitcoin

| Подход                 | Сильные стороны                           | Ограничения                                    | Контроль | Вывод                                             |
|------------------------|-------------------------------------------|------------------------------------------------|----------|---------------------------------------------------|
| Bitcoin Core RPC       | Канонический источник блоков и транзакций | Нет готовых агрегатов и управления заданиями   | высокий  | только как источник данных                        |
| electrs / Esplora      | Быстрые адресные запросы и explorer API   | Иная модель хранения, фокус на обозревателе    | средний  | полезен как аналог, но не закрывает ТЗ            |
| BlockSci               | Аналитическая модель для исследований     | Не является оперативным REST backend           | высокий  | подходит для анализа, не для эксплуатации         |
| Bitcoin ETL            | Пакетная выгрузка и подготовка датасетов  | Нет постоянного API и жизненного цикла заданий | средний  | применим для выгрузок, но не как основной backend |
| Публичные explorer API | Быстрый старт без собственного узла       | Зависимость от внешнего сервиса и лимитов      | низкий   | не подходит для ВКР                               |

Проведенное сравнение показывает, что готовые решения полезны как ориентиры, но целевая система требует собственного слоя индексации. Выбор модульного stateless-монолита позволяет сохранить простоту развертывания MVP, выделить ответственность модулей внутри одного backend-приложения и хранить состояние в PostgreSQL, а не в памяти процесса. Такой подход согласуется с требованиями к воспроизводимости, тестируемости и дальнейшему поэтапному развитию системы.

## 1.2 Понятие и особенности ончейн-данных

Под ончейн-данными в настоящей работе понимаются данные, зафиксированные в блоках канонической цепи блокчейна, а также связанные с ними производные структуры, необходимые для прикладной интерпретации состояния сети. К таким данным относятся сведения о блоках, транзакциях, входах и выходах транзакций, адресах, текущем наборе UTXO, исторических балансах и состоянии неподтвержденных транзакций в mempool.

Существенная особенность ончейн-данных заключается в их событийной природе. Состояние адресов и активов не хранится в блокчейне в виде готовых агрегатов, а восстанавливается на основе последовательной обработки транзакций и анализа связей между их входами и выходами. Это означает, что для прикладного использования данных недостаточно иметь доступ только к сырому журналу блоков: требуется дополнительный слой нормализации, агрегации и индексирования.

Другой особенностью является зависимость интерпретации данных от состояния канонической цепи. При возникновении георг часть ранее подтвержденных блоков и транзакций может потерять канонический статус, что приводит к необходимости повторной актуализации адресных индексов, агрегированных балансов и набора непотраченных выходов. Следовательно, система работы с ончейн-данными должна поддерживать не только накопление данных, но и их согласованную коррекцию.

Для прикладных сервисов особое значение имеют следующие свойства ончейн-данных:

- а) большой объем и непрерывное пополнение по мере роста цепи;
- б) необходимость восстановления производных представлений из первичных записей;
- в) чувствительность к изменению канонической ветки;
- г) различие между подтвержденными данными и временным состоянием mempool;
- д) высокая стоимость повторных выборок при отсутствии специализированных индексов.

Таким образом, ончейн-данные представляют собой не просто набор записей блокчейна, а сложный предметный массив, требующий специализированных средств структурирования, хранения и выдачи.

### 1.3 Особенности блокчейна Bitcoin как источника данных

Блокчейн Bitcoin является распределенной системой, в которой данные о транзакциях публикуются и подтверждаются в рамках механизма консенсуса. Практический доступ к этим данным в проектируемой системе осуществляется через узел Bitcoin Core по JSON-RPC. Такой способ взаимодействия обеспечивает достоверность источника, но при этом ориентирован прежде всего на обслуживание сетевого узла и администрирование, а не на выполнение аналитических или массовых прикладных выборок [1].

В качестве источника данных блокчейн Bitcoin характеризуется следующими особенностями:

- а) блочная организация данных, при которой транзакции доступны в составе последовательности блоков, связанных отношением предшествования;
- б) модель UTXO, требующая восстановления состояния адресов на основе истории создания и расходования выходов;
- в) наличие mempool как отдельного множества неподтвержденных транзакций, состояние которого изменяется независимо от подтвержденной цепи;
- г) возможность reorg, то есть пересмотра канонической ветки на ограниченной глубине;
- д) значительный объем исторических данных, требующий пакетной и устойчивой к повторной обработке индексации.

С инженерной точки зрения это означает, что прикладная система не может ограничиться единичными RPC-вызовами к узлу. Для получения данных, пригодных для дальнейшего использования, необходимо организовать полный цикл: получение блока, декомпозицию его содержимого, запись транзакций, входов и выходов в реляционную модель, обновление адресных индексов, формирование исторических агрегатов и синхронизацию с текущим состоянием mempool.

В проекте Bitcoin Blockchain Indexer эти особенности учтены через разделение контура доступа к узлу и контура прикладного хранения данных. Bitcoin Core используется как доверенный источник первичной информации, тогда как проектируемая система обеспечивает нормализацию

данных и подготовку устойчивых прикладных представлений для последующей выдачи пользователям и внешним сервисам.

## **1.4 Проблемы извлечения, хранения и предоставления ончейн-данных**

При прямой работе прикладных систем с узлом Bitcoin Core возникают трудности, связанные как с моделью хранения данных в блокчейне, так и с эксплуатационными ограничениями RPC-интерфейса. Узел предоставляет корректные первичные данные, однако не предназначен для эффективного обслуживания большого количества сложных прикладных запросов, например выборки транзакций по адресам, исторических балансов или актуальных наборов UTXO.

Основные проблемы извлечения и хранения ончейн-данных можно свести к следующим положениям:

- а) данные представлены в нормализованном для протокола, но не для прикладного использования виде;
- б) вычисление производных сущностей, таких как адресные балансы, требует последовательной обработки значительных объемов истории;
- в) повторный пересчет агрегатов при каждом пользовательском запросе приводит к неприемлемым затратам времени и ресурсов;
- г) `georg` требует корректного отката или перерасчета части ранее сохраненных представлений;
- д) `metrpool` создает отдельный класс временных данных, которые должны наблюдаться и выдаваться обособленно от подтвержденной цепи.

С точки зрения хранения значимым является вопрос согласованности. Если блок, его транзакции, набор текущих UTXO и адресные балансы обновляются несогласованно, то прикладная система теряет достоверность результатов. По этой причине система индексации должна опираться на транзакционную модель записи, идемпотентную обработку повторных событий и фиксированные правила перехода между состояниями данных.

Проблема предоставления данных также требует отдельного решения. Для внешних клиентов необходим интерфейс, который скрывает детали внутреннего формата блокчейна и предоставляет стабильные прикладные операции: получение блоков, транзакций, балансов адресов, `metrpool`-транзакций и статуса задач индексации. Следовательно,

эффективная работа с ончейн-данными предполагает наличие специализированного промежуточного слоя между блокчейн-узлом и прикладными потребителями.



## **1.5 Анализ существующих подходов и средств индексации блокчейн-данных**

Существующие подходы к работе с блокчейн-данными можно разделить на несколько групп. Первый подход заключается в прямом обращении прикладной системы к RPC интерфейсу узла. Его преимуществом является минимальный порог внедрения, однако при росте объема запросов он приводит к высокой нагрузке на узел, усложняет реализацию прикладной логики и не обеспечивает готовых индексов для быстрых выборок по адресам, транзакциям и временным диапазонам.

Второй подход основан на использовании готовых внешних блокчейн-эксплореров и публичных API. Такое решение снижает затраты на собственную инфраструктуру, но создает зависимость от стороннего поставщика данных, ограничивает контроль над полнотой и актуальностью информации, затрудняет реализацию специфических требований к безопасности и не позволяет формировать собственную модель хранения, ориентированную на прикладные сценарии.

Третий подход предполагает создание собственного индексатора, который получает данные от доверенного узла, нормализует их, сохраняет в специализированном хранилище и предоставляет прикладной API. Для задач ВКР именно этот подход является наиболее обоснованным, поскольку позволяет:

- а) контролировать полноту и достоверность источника данных;
- б) проектировать модель хранения под конкретные сценарии выборок;
- в) учитывать особенности mempool и reorg;
- г) реализовать управляемый жизненный цикл заданий индексирования;
- д) обеспечить независимость прикладных сервисов от прямого доступа к узлу.

Для сопоставления перечисленных подходов целесообразно учитывать следующие критерии: контроль над источником данных, полнота исторической информации, устойчивость к reorg, пригодность для адресной аналитики, управляемость жизненного цикла обработки и эксплуатационная независимость от внешних поставщиков. С этих позиций прямой доступ к RPC-интерфейсу целесообразен только для ограниченного круга технических задач, внешние API удобны при быстрой интеграции, а собственный

индексатор является наиболее подходящим вариантом для построения устойчивого прикладного контура данных.

Обобщенное сопоставление рассмотренных подходов приведено в таблице 3.

Таблица 3 – Сравнение подходов к получению и предоставлению блокчейн-данных

| Критерий                                  | Прямой RPC-доступ         | Внешние API           | Собственный индексатор   |
|-------------------------------------------|---------------------------|-----------------------|--------------------------|
| Контроль над данными                      | высокий                   | низкий                | высокий                  |
| Полнота исторических представлений        | ограниченная              | зависит от провайдера | высокая                  |
| Устойчивость к georg и mempool-состояниям | требует ручной реализации | зависит от провайдера | контролируется в системе |
| Пригодность для адресной аналитики        | низкая                    | средняя               | высокая                  |
| Эксплуатационная независимость            | высокая                   | низкая                | высокая                  |

В рамках рассматриваемого проекта дополнительно учитываются требования к архитектуре программной системы. В качестве архитектурного стандарта выбран модульный монолит со stateless-характером исполнения, что позволяет, с одной стороны, сохранить целостность прикладной логики в пределах одного приложения, а с другой стороны, явно разделить ответственность между модулями индексации, хранения, API, мониторинга и администрирования. Такой подход соответствует текущему масштабу разрабатываемой системы и одновременно оставляет возможности для дальнейшего расширения решения.

Следовательно, для предметной области индексации ончейн-данных оптимальным является подход, при котором система строится как специализированный программный слой между узлом Bitcoin и внешними потребителями данных.

## **1.6 Постановка задачи разработки системы индексации ончейн-данных**

На основании анализа предметной области задача настоящей выпускной квалификационной работы заключается в разработке системы индексации ончейн-данных сети Bitcoin, обеспечивающей получение информации от доверенного узла, ее нормализацию, транзакционное сохранение в реляционной базе данных и предоставление внешним потребителям через прикладной интерфейс.

Разрабатываемая система должна решать следующие основные задачи:

- а) выполнять последовательную индексацию блоков и транзакций канонической цепи;
- б) формировать и поддерживать актуальный набор UTXO, адресные балансы и их исторические представления;
- в) обрабатывать изменения состояния mempool и учитывать georg заданной глубины;
- г) предоставлять API для управления заданиями индексирования и выдачи данных;
- д) обеспечивать наблюдаемость, управляемость и пригодность к контейнерному развертыванию.

Границы проектируемой системы и ее место в общей инфраструктуре показаны на рисунке 1. Из диаграммы следует, что индексер выступает промежуточным звеном между узлом Bitcoin, административными и пользовательскими клиентами, а также подсистемой хранения данных.

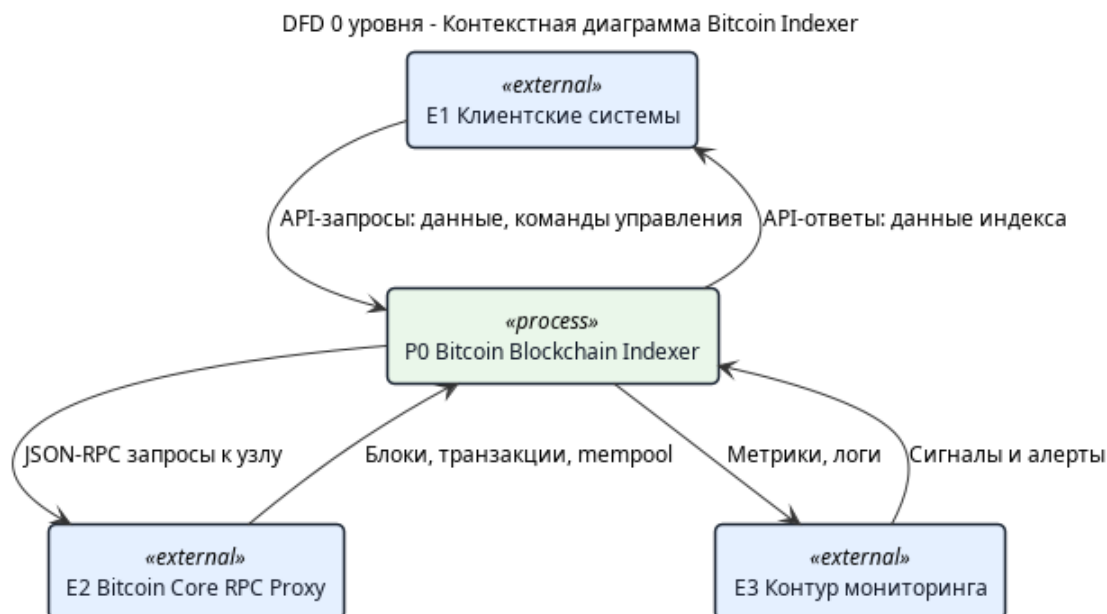


Рисунок 1 – Контекстная DFD-диаграмма системы индексации ончейн-данных

Контекстная диаграмма фиксирует внешнюю границу ответственности системы. Узел Bitcoin Core рассматривается как источник первичных блокчейн-данных, PostgreSQL — как внешнее состояние stateless-приложения, а административная панель, CLI и внешние клиенты взаимодействуют с индексатором только через прикладные интерфейсы. Такое разбиение соответствует выбранному архитектурному подходу: серверное приложение не хранит критическое состояние локально и может быть повторно запущено без потери прогресса индексации.

С учетом особенностей предметной области итоговая постановка задачи сводится к созданию программной системы Bitcoin Blockchain Indexer, обеспечивающей устойчивую, управляемую и воспроизводимую индексацию ончейн-данных сети Bitcoin, достаточную для использования в прикладных, аналитических и исследовательских сервисах. Тем самым в работе решается задача построения промежуточного инфраструктурного слоя между узлом Bitcoin и внешними потребителями данных.

## **2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ ИНДЕКСАЦИИ ОНЧЕЙН-ДАНЫХ**

После анализа предметной области следующим этапом является проектирование системы, обеспечивающей устойчивую индексацию, хранение и предоставление ончейн-данных. На данном этапе необходимо формализовать требования, определить архитектурный подход, спроектировать состав программных модулей, модель хранения данных и интерфейсы взаимодействия с внешними потребителями.

Проектирование системы Bitcoin Blockchain Indexer выполнялось с учетом ограничений предметной области и требований к эксплуатационной надежности. В качестве исходных принципов использовались модульность, управляемость жизненного цикла заданий индексирования, транзакционная согласованность данных и пригодность к контейнерному развертыванию [8; 9; 3]. Тем самым проектирование рассматривается не как произвольный выбор реализации, а как процесс обоснования архитектурных решений в соответствии с выявленными требованиями.

## **2.1 Формирование функциональных и нефункциональных требований**

Требования к проектируемой системе определяются сценариями использования ончейн-данных прикладными сервисами и особенностями самой предметной области. В частности, система должна обеспечивать получение данных от доверенного источника, воспроизводимую обработку подтвержденной цепи, корректную работу с неподтвержденными транзакциями, устойчивость к георг и удобное предоставление результатов обработки внешним системам.

Для дальнейшего проектирования требования целесообразно разделить на функциональные, требования к согласованности данных, эксплуатационные, требования к безопасности и требования к расширяемости. Такая классификация позволяет не только описать ожидаемые возможности системы, но и зафиксировать критерии, которым должны удовлетворять архитектурные и программные решения.

Таблица 4 – Основные группы требований к системе

| Группа требований            | Содержание                                                                                                                                            |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Функциональные               | Индексация блоков, транзакций, входов, выходов, UTXO, адресных балансов, mempool-данных, управление заданиями индексирования, выдача данных через API |
| Требования к согласованности | Идемпотентность записи, транзакционность операций, корректная обработка georg, детерминированное обновление производных агрегатов                     |
| Эксплуатационные             | Контейнерное развертывание, логирование, публикация метрик, восстановление после перезапуска без потери критического состояния                        |
| Требования к безопасности    | Аутентификация административных операций, защищенное подключение к RPC, управление секретами через внешнюю конфигурацию и переменные окружения        |
| Требования к расширяемости   | Возможность добавления новых модулей и развития системы без нарушения базовых контрактов взаимодействия                                               |

Функциональные и нефункциональные требования образуют основу всех дальнейших проектных решений. В частности, требование stateless-характера исполнения ведет к вынесению критического состояния в PostgreSQL, а требование воспроизводимого развертывания обосновывает использование Docker и Docker Compose [10; 11]. Требования к прикладной выдаче данных обуславливают необходимость построения специализированной модели хранения, ориентированной не только на запись блокчейн-событий, но и на быстрые выборки по адресам, транзакциям, блокам и заданиям индексирования.

Для последующего контроля полноты реализации в работе используется

матрица соответствия ключевых требований и проектных артефактов. Она позволяет разделить полностью реализованные функции, частично подтвержденные возможности и ограничения текущей версии системы.

Таблица 5 – Соответствие ключевых требований проектным артефактам

| Требование                                                         | Реализация                                                                                                | Проверяющий артефакт                  | Ст.   |
|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------|-------|
| Индексация блоков, транзакций, входов и выходов                    | Модуль <code>indexer</code> , транзакционная запись в PostgreSQL                                          | pipeline-тесты                        | да    |
| Поддержка UTXO и адресных балансов                                 | Текущие UTXO, текущие и исторические балансы адресов                                                      | миграции и data API тесты             | да    |
| Управление заданиями индексирования                                | Модуль <code>jobs</code> , REST API, CLI и административная панель                                        | jobs API тесты                        | да    |
| Обработка <code>georg</code> заданной глубины                      | Реконсиляция цепи, пометка <code>orphaned</code> , пересборка агрегатов                                   | runtime-тесты                         | да    |
| Синхронизация mempool                                              | Модуль <code>mempool</code> , статусы <code>mempool</code> и <code>dropped</code>                         | runtime-тесты                         | да    |
| REST API выдачи данных                                             | Endpoints <code>/v1/data/*</code> , фильтрация и пагинация                                                | data API тесты, OpenAPI               | да    |
| Метрики Prometheus и JSON-логирование                              | Модули <code>metrics</code> и <code>logging</code>                                                        | endpoint <code>/metrics</code> , docs | да    |
| Контейнерное развертывание                                         | <code>docker-compose.yml</code> , контейнеры <code>backend</code> , PostgreSQL и <code>admin panel</code> | README, acceptance checklist          | част. |
| HTTPS для backend API                                              | В конфигурации предусмотрены TLS-поля, но рабочий HTTPS-контур не подтвержден                             | acceptance checklist                  | нет   |
| RPC через защищенный контур <code>nginx</code> и <code>mTLS</code> | RPC-клиент поддерживает TLS/mTLS-настройки, но полный контур с <code>nginx</code> не проверен             | config/auth docs                      | част. |
| End-to-end проверка с Bitcoin Core/regtest                         | Подготовлены интеграционные сценарии с PostgreSQL и mock RPC                                              | testing docs                          | нет   |
| Покрытие backend не менее 80%                                      | Метод измерения зафиксирован через <code>cargo llvm-cov</code>                                            | coverage script                       | нет   |



---

Таким образом, проектирование системы опирается на полный набор требований MVP, однако часть эксплуатационных и проверочных требований остается ограничением текущего этапа. Это учитывается далее при анализе результатов испытаний и формировании направлений дальнейшего развития.

## **2.2 Выбор архитектурного подхода и проектирование структуры модулей**

В качестве архитектурного стандарта для разрабатываемой системы выбран модульный монолит со stateless-характером исполнения. Данный подход позволяет сохранить целостность приложения и упростить сопровождение системы, при этом разделив прикладную логику на изолированные функциональные блоки с явными границами ответственности.

Выбор именно такого архитектурного решения обусловлен следующими факторами:

- а) тесная связанность процессов индексации, хранения и выдачи данных;
- б) необходимость общей транзакционной модели работы с хранилищем;
- в) сравнительно ограниченный масштаб проекта по сравнению с распределенной микросервисной архитектурой;
- г) требование к сохранению stateless-характера приложения при хранении критического состояния во внешней базе данных;
- д) потребность в дальнейшем расширении за счет модулей, а не за счет немедленного дробления на независимые сервисы.

С инженерной точки зрения модульный монолит в рассматриваемой задаче представляет компромисс между целостностью доменной логики и требованием к локализации ответственности внутри приложения. Критическое состояние при этом хранится во внешней базе данных, что обеспечивает воспроизводимость запуска и возможность повторного развертывания экземпляра без сохранения локального сессионного состояния.

Компонентная структура проектируемой системы приведена на рисунке 2. Диаграмма отражает взаимодействие основных модулей серверной части с внешними системами и прикладными клиентами.

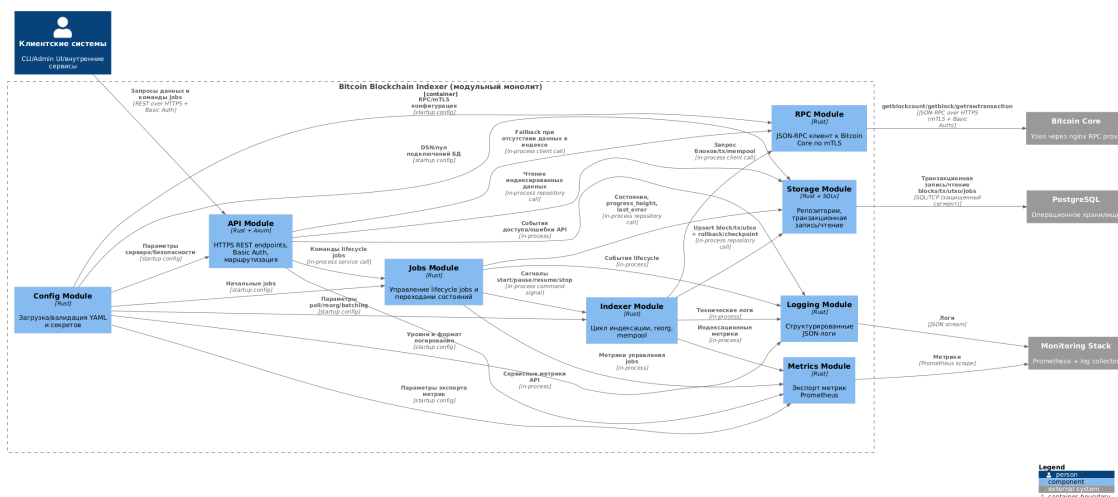


Рисунок 2 – Компонентная архитектура системы индексации ончейн-данных

Компонентная диаграмма раскрывает внутреннюю структуру этого контекста. Модули `indexer`, `mempool` и `jobs` отвечают за изменение состояния хранилища, модули `data`, `nodes` и `api` — за прикладную выдачу и управление, а `config` и `storage` обеспечивают общую инфраструктурную основу. Поэтому дальнейшее описание модели данных, интерфейсов и алгоритмов строится вокруг этих же границ ответственности.

В проектируемой системе выделены следующие основные модули:

- `config` — загрузка и валидация конфигурации, разрешение секретов;
- `api` — предоставление HTTP API и маршрутизация запросов;
- `indexer` — получение и сохранение подтвержденных блоков и транзакций;
- `jobs` — управление заданиями индексирования и их жизненным циклом;
- `mempool` — синхронизация неподтвержденных транзакций;
- `data` — прикладные выборки из хранилища;
- `nodes` — мониторинг состояния подключенных узлов;
- `storage` — репозитории и транзакционная работа с PostgreSQL.

Такое разбиение позволяет локализовать ответственность каждого модуля, обеспечить независимое развитие отдельных частей системы и минимизировать влияние локальных изменений на общую архитектуру. Тем самым структура модулей не только отражает текущую реализацию, но и служит средством обеспечения расширяемости и сопровождаемости

программного комплекса.

## 2.3 Проектирование модели данных и схемы хранения

Для реализации функций индексирования и прикладной выдачи данных в проекте используется реляционная модель хранения на базе PostgreSQL. Выбор данного подхода обусловлен необходимостью обеспечения транзакционной согласованности, поддержки ограничений целостности, эффективной индексации и формирования многообразных выборок по связанным сущностям [3].

Проектируемая схема хранения включает несколько классов сущностей:

- а) сущности первичного блокчейн-уровня: блоки, транзакции, входы и выходы;
- б) сущности текущего состояния: набор UTXO, текущие балансы адресов;
- в) сущности исторических представлений: история балансов адресов;
- г) сущности управления системой: задания индексирования и связанные с ними адресные фильтры;
- д) сущности наблюдаемости: сведения о состоянии подключенных узлов.

В таблице 7 представлены ключевые проектируемые таблицы, их основные ограничения и роль в процессе индексации.

Таблица 7 – Ключевые элементы модели данных

| Таблица                 | Ключи и индексы                                          | Основные поля                          | Роль в системе                                   |
|-------------------------|----------------------------------------------------------|----------------------------------------|--------------------------------------------------|
| blocks                  | PK id, unique hash, индексы по высоте и статусу          | height, hash, prev_hash, status        | фиксация канонических и неканонических блоков    |
| transactions            | PK id, unique txid, индексы по статусу, высоте и времени | txid, block_hash, block_height, status | связь транзакций с блоками и mempool-состояниями |
| tx_outputs              | PK (txid, vout), индекс по адресу                        | txid, vout, address, value_sats        | база для адресных выборок и расчета UTXO         |
| tx_inputs               | PK (txid, vin), индекс по расходуемому выходу            | txid, vin, prev_txid, prev_vout        | фиксация расходов                                |
| utxos current           | PK (out_txid, out_vout), индекс по адресу и статусу      | address, value_sats, status            | актуальное состояние непотраченных выходов       |
| address balance current | PK address                                               | confirmed_sats, updated_height         | быстрый ответ по текущему балансу адреса         |

## Продолжение таблицы 7

| Таблица                       | Ключи и индексы                                        | Основные поля                             | Роль в системе                             |
|-------------------------------|--------------------------------------------------------|-------------------------------------------|--------------------------------------------|
| address<br>balance<br>history | PK (address, block_height), индекс по адресу и времени | balance_sats, block_height, block_time    | история изменения адресного баланса        |
| jobs                          | PK job_id, CHECK по статусам                           | mode, status, from_height, current_height | управление жизненным циклом индексирования |
| job<br>addresses              | PK (job_id, address), FK на jobs                       | job_id, address                           | адресные фильтры для выборочных заданий    |
| node_health                   | PK node_id                                             | last_seen_at, tip_height, status          | наблюдение за доступностью и высотой узлов |

Важным свойством модели данных является ее ориентированность на идемпотентность и воспроизводимость обработки. Для этого в проекте используются первичные и уникальные ключи, ограничения целостности и детерминированные правила обновления агрегатов. С инженерной точки зрения это позволяет описать лаг индексации как разность между высотой tip цепи и текущим прогрессом job:

$$L = H_{tip} - H_{progress}, \quad (1)$$

где  $L$  — лаг индексирования,  $H_{tip}$  — актуальная высота цепи,  $H_{progress}$  — высота, до которой синхронизирован job.

Данная модель хранения обеспечивает как последовательную обработку блоков, так и эффективное выполнение прикладных запросов к уже подготовленным данным, что является ключевым требованием к системе индексации ончейн-данных.

## 2.4 Проектирование интерфейсов взаимодействия и сценариев работы системы

Интерфейсы проектируемой системы строятся вокруг трех основных контуров взаимодействия: контура получения данных от узла Bitcoin, контура административного управления и контура прикладной выдачи данных внешним потребителям. Такое разделение позволяет формализовать назначение каждой группы операций и уменьшить связанность между внешними участниками системы.

Контур получения данных ориентирован на доверенное взаимодействие с Bitcoin Core через RPC-прокси. Он предназначен исключительно для внутреннего использования индексером и не должен быть доступен прикладным клиентам. Контур административного управления реализуется через REST API, административную панель и CLI. В его рамках выполняются операции запуска, остановки, паузы и возобновления заданий индексирования, а также получение сведений о состоянии узлов и выполняемых процессов. Контур прикладной выдачи предоставляет доступ к блокам, транзакциям, балансам адресов, UTXO и mempool-данным.

Основные проектируемые группы REST endpoints включают:

- а) endpoints управления заданиями индексирования;
- б) endpoints мониторинга и диагностики узлов;
- в) endpoints получения блоков, транзакций, балансов и UTXO;
- г) сервисные endpoints наблюдаемости и документирования API.

В таблице 9 приведены основные группы прикладных endpoint'ов и их назначение.

Таблица 9 – Группы REST API системы

| Группа    | Endpoint'ы                                                                | Назначение                                                     |
|-----------|---------------------------------------------------------------------------|----------------------------------------------------------------|
| Jobs API  | /v1/jobs, /v1/jobs/{id},<br>операции start, stop, pause,<br>resume, retry | создание, просмотр и<br>управление заданиями<br>индексирования |
| Nodes API | /v1/nodes,<br>/v1/nodes/{id}                                              | получение сведений о<br>подключенных узлах и их<br>состоянии   |
| Data API  | /v1/data/addresses/*,<br>/v1/data/transactions,<br>/v1/data/blocks        | прикладная выдача<br>балансов, UTXO,<br>транзакций и блоков    |

## Продолжение таблицы 9

| Группа        | Endpoint'ы                     | Назначение                                            |
|---------------|--------------------------------|-------------------------------------------------------|
| Observability | /metrics, OpenAPI-документация | мониторинг и машинно-читаемое описание контрактов API |

Жизненный цикл задания индексирования формализуется набором допустимых переходов, приведенных в таблице 11. Такая фиксация важна для единообразного поведения REST API, CLI и административной панели.

Таблица 11 – Допустимые переходы состояний job

| Операция | Переход                          | Условие                              | Результат                                          |
|----------|----------------------------------|--------------------------------------|----------------------------------------------------|
| start    | created -> running               | job создан и разрешен к запуску      | runner начинает обработку высот                    |
| pause    | running -> paused                | job находится в работе               | прогресс сохраняется, обработка приостанавливается |
| resume   | paused -> running                | job был поставлен на паузу           | обработка продолжается с сохраненной высоты        |
| stop     | running paused failed -> created | требуется сброс активного выполнения | job возвращается в исходное состояние              |
| retry    | failed -> running                | ошибка устранена оператором          | job запускается повторно с сохраненным прогрессом  |

Проектирование интерфейсов опирается на единый набор принципов:

- предоставление данных в прикладной форме, а не в сыром RPC-представлении;
- детерминированные форматы ответов и ошибок;
- разграничение подтвержденных данных и metpool-представлений;
- защита административных функций средствами аутентификации;
- пригодность интерфейсов для использования как человеком, так и внешними системами.



Сценарии функционирования системы включают последовательную индексацию цепи, обновление адресных агрегатов, обслуживание пользовательских запросов и административное управление ходом обработки. Тем самым интерфейсы проектируются не как набор разрозненных конечных точек доступа, а как формализованное средство реализации основных процессов системы. В дальнейшем именно эти сценарии становятся основой для реализации программных модулей и для построения программы испытаний разработанного программного средства.

### 3 РАЗРАБОТКА СИСТЕМЫ ИНДЕКСАЦИИ ОНЧЕЙН-ДАННЫХ

#### 3.1 Реализация серверной части системы

Практическая реализация проектируемой системы выполнена в виде серверного приложения на языке Rust. Серверная часть объединяет загрузку конфигурации, инициализацию подключений к базе данных и RPC-источнику, запуск HTTP API, фоновых процессов обработки и средств наблюдаемости. Такой способ организации соответствует выбранной архитектуре модульного монолита со stateless-характером исполнения и обеспечивает единый жизненный цикл всех компонентов приложения.

Ключевая роль серверной части заключается в координации нескольких взаимосвязанных процессов:

- а) приема и валидации конфигурации запуска;
- б) синхронизации заданий индексирования из YAML-конфигурации и управляющего API;
- в) выполнения индексации подтвержденных блоков и транзакций;
- г) отдельной синхронизации состояния mempool;
- д) публикации API, метрик и диагностической информации.

С инженерной точки зрения такое построение позволяет рассматривать систему как единый прикладной контур обработки данных, в котором критическое состояние сохраняется во внешней базе данных, а экземпляр приложения может быть перезапущен без потери консистентности. Тем самым реализация серверной части непосредственно поддерживает сформулированные ранее требования к восстанавливаемости и управляемости системы.

Ключевые инженерные решения, использованные при реализации серверной части, приведены в таблице 13. Они связывают архитектурные требования с конкретными механизмами кода и базы данных.

Таблица 13 – Ключевые инженерные решения реализации

| Решение                         | Назначение                                        | Реализация                                           | Проверка       |
|---------------------------------|---------------------------------------------------|------------------------------------------------------|----------------|
| Advisory lock на состояние цепи | исключить параллельную запись конфликтующих высот | advisory lock в PostgreSQL в транзакции записи блока | pipeline-тесты |

### Продолжение таблицы 13

| Решение                          | Назначение                                                 | Реализация                                 | Проверка       |
|----------------------------------|------------------------------------------------------------|--------------------------------------------|----------------|
| Идемпотентная запись             | безопасно повторять обработку блока или job                | ON CONFLICT, первичные и уникальные ключи  | storage-тесты  |
| Разделение статусов транзакций   | не смешивать confirmed, mempool, dropped и orphaned данные | поле status и отдельные API-фильтры        | data API тесты |
| Пересборка агрегатов после georg | восстановить UTXO и балансы по канонической ветке          | пометка orphaned и replay canonical blocks | georg-тесты    |
| Прогресс jobs в БД               | сохранить управляемость после рестарта приложения          | поле progress_height и переходы job        | jobs API тесты |
| Метрики Prometheus               | наблюдать лаг, ошибки и объем обработки                    | /metrics, счетчики и gauge-показатели      | smoke-проверка |

## 3.2 Реализация механизма индексации блоков и транзакций

Основной процесс обработки подтвержденных данных реализован в модуле индексатора. Он получает данные о блоках от Bitcoin Core, выполняет декомпозицию транзакций, входов и выходов, после чего сохраняет результат в PostgreSQL в рамках одной транзакции. Такой подход обеспечивает атомарность записи и предотвращает частичное появление несогласованных данных.

Алгоритм пакетной индексации подтвержденных данных представлен на рисунке 3. В нем отражены ключевые стадии: определение диапазона высот, проверка расхождения цепи, последовательная запись блоков и обновление прогресса задания индексирования.

**Исходные параметры:** конфигурация job, состояние БД,  
RPC-клиент

**Результат:** обновленные данные подтвержденной цепи и прогресс  
задания

- 1 получить текущую высоту узла;
- 2 определить диапазон высот для следующего батча;
- 3 проверить расхождение цепи в окне `reorg_depth`;
- 4 **если обнаружен reorg тогда**
  - 5 пометить неканонические блоки и транзакции;
  - 6 пересобрать агрегаты UTXO и балансов;
  - 7 откатить `progress_height`;
- 8 **для каждого высота в выбранном диапазоне выполнять**
  - 9 получить блок по RPC;
  - 10 декомпозировать транзакции, входы и выходы;
  - 11 сохранить блок и связанные сущности в одной транзакции БД;
  - 12 обновить UTXO и баланс адресов;
  - 13 обновить `progress_height`;

Рисунок 3 – Алгоритм пакетной индексации данных  
подтвержденной цепи

Особое внимание в реализации уделено идемпотентности. Повторная обработка уже сохраненной высоты не должна приводить к появлению дубликатов или искажению адресных агрегатов. Для этого используются ограничения целостности базы данных, транзакционная запись и согласованное обновление производных сущностей. Таким образом, реализация индексатора обеспечивает не только накопление данных, но и воспроизводимость результата обработки.

### 3.3 Реализация управления заданиями индексирования, `metpool` и `reorg`

Управление заданиями индексирования реализовано как отдельный функциональный контур, обеспечивающий хранение конфигурации заданий, контроль их состояния и периодический запуск батчей индексации. Каждое

задание имеет собственный идентификатор, режим работы, текущий статус, информацию о прогрессе и диагностические данные о последней ошибке. Это позволяет использовать задание как единицу управляемости и наблюдаемости процесса индексации.

Наряду с обработкой подтвержденной цепи в системе реализован отдельный контур синхронизации mempool. Его задача состоит в периодическом опросе узла, сохранении неподтвержденных транзакций и пометке исчезнувших записей как dropped. При этом агрегаты подтвержденной цепи не смешиваются с mempool, поскольку их семантика принципиально различается.

Алгоритм синхронизации mempool представлен на рисунке 4.

**Исходные параметры:** список известных mempool tx, ответ

getrawmempool

**Результат:** актуальное состояние mempool в БД

- 1 получить текущий список txid из mempool;
- 2 определить новые и исчезнувшие транзакции;
- 3 **для каждого нового txid выполнять**
  - 4 запросить verbose-представление транзакции;
  - 5 сохранить запись со статусом mempool;
  - 6 сохранить входы и выходы для будущей фильтрации по адресу;
- 7 **для каждого исчезнувшего txid выполнять**
  - 8 пометить запись как dropped, если она не подтверждена;

Рисунок 4 – Алгоритм синхронизации mempool

Для обеспечения корректности данных канонической цепи в системе реализована обработка georg. При обнаружении расхождения между сохраненной в БД цепью и актуальным состоянием узла неканонические блоки переводятся в статус orphaned, затронутые транзакции получают соответствующую маркировку, а производные агрегаты пересобираются по оставшейся канонической ветке. Одновременно прогресс задания индексирования откатывается до согласованной высоты, после чего индексация продолжается уже по новой ветви цепи.

### 3.4 Реализация API, конфигурирования и мониторинга

Внешнее взаимодействие с системой реализовано через REST API, которое предоставляет операции управления заданиями индексирования, диагностики узлов и получения проиндексированных данных. Для сопровождения системы предусмотрены также административная панель и CLI, работающие поверх того же HTTP-контракта.

Конфигурирование серверного приложения выполняется через YAML-файл и переменные окружения. В конфигурации задаются параметры API, настройки подключения к RPC-узлу, ограничения параллелизма, параметры заданий индексирования и режимы обработки mempool и georg. Такой подход делает конфигурацию внешней по отношению к приложению и хорошо согласуется с требованиями контейнерного развертывания.

Наблюдаемость системы обеспечивается за счет JSON-логирования и публикации метрик Prometheus. В частности, экспортируются показатели текущей высоты цепи, прогресса заданий индексирования, лага индексации, числа обработанных блоков и транзакций, латентности RPC-вызовов и ошибок ключевых подсистем. Это позволяет использовать разработанную систему как основу для реального эксплуатационного контура и подтверждает соответствие реализованных средств сопровождения исходным эксплуатационным требованиям.

### 3.5 Кодовые фрагменты, иллюстрирующие реализацию

Для подтверждения того, что изложенные проектные решения реализованы в кодовой базе, в работе приведены листинги ключевых фрагментов программной системы. Они выбраны не по признаку краткости, а по связи с критическими требованиями: транзакционная согласованность, идемпотентность обработки и управляемость жизненного цикла заданий индексирования.

На рисунке 5 приведен фрагмент метода сохранения блока. Он показывает начало транзакционной записи, использование advisory lock для синхронизации состояния цепи, проверку повторной обработки уже сохраненной высоты и ожидание предыдущего блока перед записью следующей высоты. Данный фрагмент непосредственно связан с требованиями корректной обработки подтвержденной цепи, идемпотентности и устойчивости к повторному запуску job.

```
1  pub async fn persist_block(&self, block: &RpcBlock) -> Result<
    PersistBlockOutcome, sqlx::Error> {
2      let mut db_tx = self.pool.begin().await?;
3      acquire_chain_state_lock(&mut *db_tx).await?;
4      acquire_height_lock(&mut *db_tx, block.height).await?;
5
6      if let Some(existing_hash) = canonical_block_hash_at_height(&mut *db_tx,
    block.height).await? {
7          db_tx.commit().await?;
8          if existing_hash == block.hash {
9              return Ok(PersistBlockOutcome::AlreadyIndexed);
10         }
11
12         return Err(sqlx::Error::Protocol(format!(
13             "height {} is already occupied by canonical block {}",
14             block.height, existing_hash
15         )));
16     }
17
18     if block.height > 0 && canonical_block_hash_at_height(&mut *db_tx, block.
    height - 1).await?.is_none() {
19         db_tx.commit().await?;
20         return Ok(PersistBlockOutcome::WaitingForPreviousHeight);
```

Рисунок 5 – Фрагмент транзакционной и идемпотентной записи блока

На рисунке 6 приведена функция, задающая допустимые переходы

состояний job. Именно этот фрагмент делает жизненный цикл задания формализованным контрактом: недопустимые переходы возвращают ошибку, а REST API, CLI и административная панель используют единую бизнес-логику управления индексированием.

```
1 fn transition_target(action: JobAction, current: &str) -> Result<&'static str,  
    JobsError> {  
2     match (action, current) {  
3         (JobAction::Start, "created") => Ok("running"),  
4         (JobAction::Stop, "running") => Ok("created"),  
5         (JobAction::Stop, "paused") => Ok("created"),  
6         (JobAction::Stop, "failed") => Ok("created"),  
7         (JobAction::Pause, "running") => Ok("paused"),  
8         (JobAction::Resume, "paused") => Ok("running"),  
9         (JobAction::Retry, "failed") => Ok("running"),  
10        _ => Err(JobsError::InvalidTransition(current.to_string())),  
11    }
```

Рисунок 6 – Фрагмент проверки допустимых переходов job

Командный интерфейс реализован в отдельном модуле cli/indexer\_cli.py и используется для вспомогательного взаимодействия с заданиями индексирования, узлами и прикладными данными через тот же REST API, что и административная панель.



## **4 ИССЛЕДОВАНИЕ И ОЦЕНКА КАЧЕСТВА РАЗРАБОТАННОЙ СИСТЕМЫ**

### **4.1 Цели и задачи испытаний**

После реализации программной системы необходима проверка того, что полученное решение соответствует сформулированным требованиям и сохраняет корректность работы в ключевых для предметной области сценариях. Для системы индексации ончейн-данных такая проверка должна охватывать не только общие API-функции, но и устойчивость к георг, корректность работы с mempool, согласованность хранилища и работоспособность управляющих механизмов.

Целью испытаний является подтверждение пригодности разработанной системы Bitcoin Blockchain Indexer к решению задач индексации, хранения и выдачи ончейн-данных. Для достижения этой цели необходимо решить следующие задачи:

- а) проверить корректность функций управления заданиями индексирования и взаимодействия с API;
- б) проверить согласованность записи подтвержденных данных в хранилище;
- в) оценить корректность обработки mempool и реорганизаций цепи;
- г) подтвердить работоспособность механизмов мониторинга и диагностики;
- д) сопоставить фактические результаты с установленными критериями проверки.

### **4.2 Программа и методика испытаний**

Испытания системы строятся на сочетании интеграционного тестирования, функциональной проверки API и анализа критериев приемки. В качестве основного подхода используются тесты, запускаемые в окружении с контейнером PostgreSQL через testcontainers, что позволяет приблизить условия проверки к реальной эксплуатации, сохраняя воспроизводимость тестовых сценариев [12].

В рамках программы испытаний проверяются следующие группы сценариев:

- а) сценарии жизненного цикла заданий индексирования: создание, запуск, пауза, возобновление, остановка и обработка ошибочных переходов;
- б) сценарии API nodes и data API, включая запросы балансов, UTXO, транзакций, блоков и mempool-данных;
- в) сценарии работы контура индексации и проверки идемпотентности записи подтвержденных блоков;
- г) сценарии фоновых процессов обработки, включая синхронизацию mempool и обработку georg;
- д) сценарии наблюдаемости, связанные с публикацией метрик и диагностической информации о состоянии узлов и заданий индексирования.

Для фиксации результатов испытаний используется протокол, приведенный в таблице 15. Он отделяет реализованные тестовые сценарии от тех проверок, которые требуют отдельного запуска или дополнительного эксплуатационного окружения.

Таблица 15 – Протокол проверки качества системы

| Проверка                                   | Окружение и артефакт                            | Статус                | Ограничение                                  |
|--------------------------------------------|-------------------------------------------------|-----------------------|----------------------------------------------|
| Jobs lifecycle и ошибки переходов          | PostgreSQL через testcontainers, jobs API tests | сценарии реализованы  | требуется ручной запуск ignored-тестов       |
| Nodes API и диагностика узлов              | интеграционные API-тесты                        | сценарии реализованы  | данные узла моделируются тестовым состоянием |
| Data API: балансы, UTXO, транзакции, блоки | PostgreSQL, миграции, тестовый AppState         | сценарии реализованы  | используются подготовленные тестовые данные  |
| Storage pipeline и идемпотентность         | pipeline-тесты и тестовые блоки                 | сценарии реализованы  | источник RPC моделируется в тесте            |
| Mempool и georg                            | runtime runner-тесты, mock JSON-RPC             | частично подтверждено | нет запуска с реальным Bitcoin Core/regtest  |
| Метрики Prometheus                         | endpoint /metrics и smoke-проверка формата      | частично подтверждено | нет длительного эксплуатационного стенда     |

Продолжение таблицы 15

| Проверка         | Окружение и артефакт             | Статус             | Ограничение                                  |
|------------------|----------------------------------|--------------------|----------------------------------------------|
| Покрытие backend | coverage-скрипт и cargo llvm-cov | метод зафиксирован | фактический порог 80% не подтвержден отчетом |

Методически испытания ориентированы на подтверждение ключевых свойств системы: функциональной корректности, согласованности данных, восстанавливаемости после сбоев и эксплуатационной наблюдаемости. Для итоговой оценки используется перечень критериев приемки, фиксирующий степень выполнения отдельных требований.

#### 4.3 Проверка корректности функциональных возможностей

Функциональная корректность системы подтверждается набором интеграционных тестов и реализованными прикладными сценариями взаимодействия с API. Проверяются операции управления заданиями индексирования, получение списка узлов, запросы подтвержденных транзакций, балансов адресов, UTXO и блоков, а также пограничные ситуации, связанные с ошибками авторизации, отсутствием объекта или нарушением ограничений переходов состояний.

С точки зрения предметной области особенно важны результаты проверки следующих свойств:

- а) задания индексирования обладают формализованным жизненным циклом и корректно меняют состояние через API;
- б) подтвержденные данные сохраняются в БД в виде связанных сущностей: блоков, транзакций, входов, выходов и агрегатов адресного уровня;
- в) mempool-данные доступны как отдельный класс сущностей и не смешиваются с представлениями подтвержденной цепи;
- г) выдача данных через API ориентирована на прикладные выборки, а не на сырые RPC-ответы.

Таким образом, функциональное тестирование показывает, что разработанная система реализует основной набор возможностей, предусмотренный сформулированными требованиями к программному

комплексу, и обеспечивает прикладную форму доступа к данным, отличную от прямой работы с RPC узла.

#### **4.4 Оценка устойчивости к georg и изменениям состояния mempool**

Для систем индексации ончейн-данных устойчивость к изменениям состояния сети является одним из ключевых критериев качества. В рассматриваемом проекте это свойство проверяется через сценарии, связанные с georg и динамикой mempool.

Испытания georg подтверждают, что при обнаружении расхождения между сохраненной в БД цепью и актуальными данными узла система:

- а) помечает затронутые блоки как неканонические;
- б) переводит связанные транзакции в статус orphaned;
- в) пересчитывает производные агрегаты по оставшейся канонической ветке;
- г) откатывает прогресс задания индексирования до согласованной высоты.

Испытания mempool подтверждают, что фоновый процесс синхронизации корректно обрабатывает появление новых неподтвержденных транзакций и помечает исчезнувшие как dropped. При этом текущие балансы подтвержденной цепи и подтвержденные UTXO не искажаются за счет временного состояния mempool. Такое разделение представлений принципиально важно для достоверности прикладных ответов.

Следовательно, система демонстрирует устойчивость к двум наиболее характерным для предметной области видам изменчивости: локальному пересмотру канонической цепи и колебаниям состава неподтвержденных транзакций.

#### **4.5 Оценка эксплуатационных характеристик и наблюдаемости**

Помимо функциональной корректности, для практического использования системы важны ее эксплуатационные свойства. В разработанном решении они обеспечиваются через контейнеризованное развертывание, конфигурирование из внешних источников, JSON-логирование и публикацию метрик Prometheus [13].

К числу значимых эксплуатационных характеристик относятся:

- а) воспроизводимость запуска серверного приложения и базы данных в контейнерном окружении;
- б) возможность мониторинга прогресса заданий индексирования и текущей высоты цепи;
- в) наблюдение за RPC-запросами, задержками и ошибками записи в БД;
- г) наличие диагностической информации по состоянию узлов;
- д) пригодность системы к сопровождению через API, CLI и административную панель.

В проекте экспортируются метрики текущей высоты цепи, прогресса заданий индексирования, лага индексации, количества обработанных блоков и транзакций, а также статистика RPC-запросов и ошибок. Это позволяет использовать систему как основу для последующей эксплуатационной интеграции.

#### **4.6 Анализ результатов испытаний**

Анализ выполненных испытаний показывает, что основная часть требований к системе реализована и подтверждена кодом, интеграционными тестами и критериями приемки. Наиболее полно подтверждены работоспособность схемы хранения, жизненный цикл заданий индексирования, корректность записи подтвержденных данных, обработка georg, разделение mempool и подтвержденной цепи, а также наличие REST API, CLI и административной панели.

Вместе с тем анализ результатов испытаний показывает наличие направлений, требующих дальнейшего расширения экспериментальной проверки. К ним относятся подтверждение защищенного эксплуатационного контура взаимодействия, полномасштабные end-to-end испытания с реальным Bitcoin Core/regtest и более детальная фиксация итоговых показателей покрытия серверной части тестами. Указанные вопросы не ставят под сомнение достигнутую работоспособность системы, однако определяют перспективы углубления ее верификации.

В целом результаты испытаний позволяют сделать вывод о том, что разработанная система успешно решает основную задачу индексации ончейн-данных, а выбранные архитектурные и проектные решения являются обоснованными с точки зрения предметной области и дальнейшего развития проекта.

## 5 ПРАКТИЧЕСКАЯ ЗНАЧИМОСТЬ И ПЕРСПЕКТИВЫ РАЗВИТИЯ СИСТЕМЫ

### 5.1 Практическая значимость разработанного решения

Практическая значимость разработанной системы определяется тем, что она формирует специализированный слой доступа к ончейн-данным Bitcoin, пригодный для использования прикладными и аналитическими сервисами. В отличие от прямой работы с узлом сети, такое решение предоставляет уже нормализованные и подготовленные данные, что снижает сложность интеграции и уменьшает нагрузку на инфраструктуру источника данных.

Для практического применения наиболее важны следующие свойства разработанной системы:

- а) наличие прикладного API для доступа к блокам, транзакциям, UTXO, балансам адресов и состоянию заданий индексирования;
- б) возможность управляемой индексации с контролем прогресса и ошибок;
- в) наличие административных и вспомогательных средств сопровождения;
- г) готовность к контейнерному разворачиванию и мониторингу;
- д) сохранение согласованности данных подтвержденной цепи при georg и при работе с mempool.

Конкретные пользовательские сценарии, в которых проявляется практическая ценность системы, приведены в таблице 17. Они показывают, какие интерфейсы и данные используются для решения типовых прикладных задач.

Таблица 17 – Практические сценарии применения системы

| Сценарий                          | API                                                                                         | Данные БД                                                 | Практический результат и отличие от RPC                                                                              |
|-----------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Получение текущего баланса адреса | GET /v1/data/addresses/{address}/balance                                                    | текущий баланс адресов, блоки                             | сервис сразу возвращает сумму в satoshi и высоту среза; клиенту не нужно самостоятельно обходить UTXO через RPC узла |
| Анализ UTXO и истории баланса     | GET /v1/data/addresses/{address}/utxos;<br>GET /v1/data/addresses/{address}/balance/history | текущие UTXO, история балансов, входы и выходы транзакций | прикладной сервис получает готовые срезы состояния адреса, а не реконструирует историю по сырым блокам               |

## Продолжение таблицы 17

| Сценарий                              | API                                                     | Данные БД                                           | Практический результат и отличие от RPC                                                                                         |
|---------------------------------------|---------------------------------------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Мониторинг прогресса full-sync        | GET /v1/jobs;<br>GET /v1/jobs/{job_id};<br>GET /metrics | задания индексирования, состояние узлов, метрики    | оператор видит статус, прогресс и ошибки индексирования; Bitcoin Core RPC не хранит состояние прикладного процесса индексации   |
| Просмотр mempool-транзакций по адресу | GET /v1/data/transactions/mempool?address={address}     | транзакции, входы, выходы и адресный индекс mempool | неподтвержденные транзакции отделены от confirmed-цепи и доступны с адресным фильтром без ручной обработки ответа getrawmempool |

### 5.2 Возможности применения системы в прикладных сервисах

Разработанная система может использоваться в составе различных программных решений, работающих с блокчейн-данными Bitcoin. В частности, она подходит для:

- а) аналитических сервисов, выполняющих выборки по адресам, транзакциям и блокам;
- б) административных и мониторинговых панелей, отображающих состояние узлов и заданий индексирования;
- в) внутренних корпоративных сервисов, которым требуется контролируемый доступ к ончейн-данным без прямого обращения к RPC узла;
- г) исследовательских и учебных стендов, демонстрирующих процессы индексации, хранения и предоставления блокчейн-данных.

Наличие CLI и административной панели расширяет область практического использования системы, поскольку позволяет применять ее как в автоматизированных сценариях, так и в задачах оперативного сопровождения.

### 5.3 Ограничения текущей версии системы

Несмотря на достижение основной цели работы, текущая версия системы имеет ряд ограничений. К ним относятся:

- а) неполное подтверждение end-to-end сценариев с реальным Bitcoin

Core;

- б) необходимость дальнейшего подтверждения защищенного эксплуатационного контура взаимодействия;
- в) ограниченная глубина экспериментальной оценки производительности;
- г) применение общей стратегии индексации без специализированной оптимизации для отдельных пользовательских режимов, например `address_list`;
- д) отсутствие расширенного эксплуатационного alerting и более детальных сценариев деградации узлов.

Указанные ограничения не снижают значимости достигнутого результата, но определяют границы текущего этапа разработки и подтверждают, что система находится на стадии, ориентированной на дальнейшее инженерное развитие.

#### **5.4 Направления дальнейшего развития системы**

Перспективы развития разработанной системы связаны как с углублением функциональности, так и с расширением эксплуатационных возможностей. Наиболее естественными направлениями дальнейшей работы являются:

- а) проведение полноценных end-to-end испытаний с реальным узлом Bitcoin Core;
- б) развитие защищенного контура взаимодействия серверной части и внешних клиентов;
- в) оптимизация производительности индексирования и прикладных выборок;
- г) расширение режимов работы заданий индексирования и стратегий фильтрации по адресам;
- д) развитие средств мониторинга, диагностики и автоматического реагирования;
- е) подготовка архитектурной базы для дальнейшего масштабирования решения.

С учетом выбранного архитектурного подхода указанные направления могут быть реализованы поэтапно без отказа от уже созданной модели хранения и структуры основных модулей.



## **5.5 Выводы по результатам работы**

Разработанная система индексации ончейн-данных обладает практической ценностью как самостоятельный программный компонент, обеспечивающий контролируемое получение, хранение и выдачу блокчейн-данных Bitcoin. Ее применение позволяет снизить сложность интеграции прикладных сервисов с блокчейн-инфраструктурой и создает основу для дальнейшего развития в сторону более полного промышленного решения.

## ЗАКЛЮЧЕНИЕ

В выпускной квалификационной работе решена задача проектирования и разработки системы индексации ончейн-данных сети Bitcoin. В процессе выполнения работы были последовательно рассмотрены теоретические основы предметной области, проанализированы особенности блокчейна Bitcoin как источника данных, сформированы требования к системе, спроектированы архитектура программного комплекса и модель хранения, а также описана реализация ключевых модулей индексации, управления, мониторинга и прикладной выдачи данных.

В результате выполненной работы создан программный комплекс Bitcoin Blockchain Indexer, реализующий индексацию блоков и транзакций, поддержку текущего состояния UTXO, адресных балансов, обработку mempool и georg, управление заданиями индексирования, а также предоставление данных через REST API, административную панель и CLI. Проведенные испытания подтвердили работоспособность основных функций системы в рамках проверенных интеграционных сценариев и показали обоснованность выбранных архитектурных и проектных решений для уровня MVP.

Практическая значимость работы состоит в создании основы для дальнейшего развития инфраструктурного решения, предназначенного для использования в прикладных и аналитических сервисах, работающих с ончейн-данными Bitcoin. Полученные результаты могут быть использованы как для дальнейшей инженерной доработки проекта, так и для развития исследований в области системной индексации блокчейн-данных.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Bitcoin Core RPC Documentation. — URL: <https://developer.bitcoin.org/reference/rpc/index.html> (дата обращения 22.03.2026).
2. Nakamoto S. Bitcoin: A Peer-to-Peer Electronic Cash System. — 2008. — URL: <https://bitcoin.org/bitcoin.pdf> (дата обращения 22.03.2026).
3. PostgreSQL 16 Documentation. — URL: <https://www.postgresql.org/docs/16/index.html> (дата обращения 22.03.2026).
4. electrs: An efficient re-implementation of Electrum Server in Rust. — URL: <https://github.com/romanz/electrs> (дата обращения 20.04.2026).
5. Esplora Block Explorer. — URL: <https://github.com/Blockstream/esplora> (дата обращения 20.04.2026).
6. BlockSci: Design and Applications of a Blockchain Analysis Platform / Н. Kalodner [и др.] // — 2017. — URL: <https://collaborate.princeton.edu/en/publications/blocksci-design-and-applications-of-a-blockchain-analysis-platfor/> (дата обращения 20.04.2026).
7. Bitcoin ETL. — URL: <https://github.com/blockchain-etl/bitcoin-etl> (дата обращения 20.04.2026).
8. ГОСТ Р 57100-2016. Системная и программная инженерия. Описание архитектуры. — Стандартинформ, 2016.
9. ГОСТ Р ИСО/МЭК 25010-2015. Модели качества систем и программных продуктов. — Стандартинформ, 2015.
10. Docker Documentation. — URL: <https://docs.docker.com/> (дата обращения 22.03.2026).
11. Docker Compose Documentation. — URL: <https://docs.docker.com/compose/> (дата обращения 22.03.2026).
12. testcontainers for Rust. — URL: <https://docs.rs/testcontainers/> (дата обращения 22.03.2026).
13. Prometheus Documentation. — URL: <https://prometheus.io/docs/introduction/overview/> (дата обращения 22.03.2026).

## ПРИЛОЖЕНИЕ А

### Диаграмма контейнерного развёртывания

В приложении приведена схема контейнерного развёртывания проекта. Рисунок А.1 использует подготовленный в каталоге `report` архитектурный артефакт и показывает состав runtime-окружения: backend, PostgreSQL, административную панель, прокси-слой и внешний Bitcoin RPC узел.

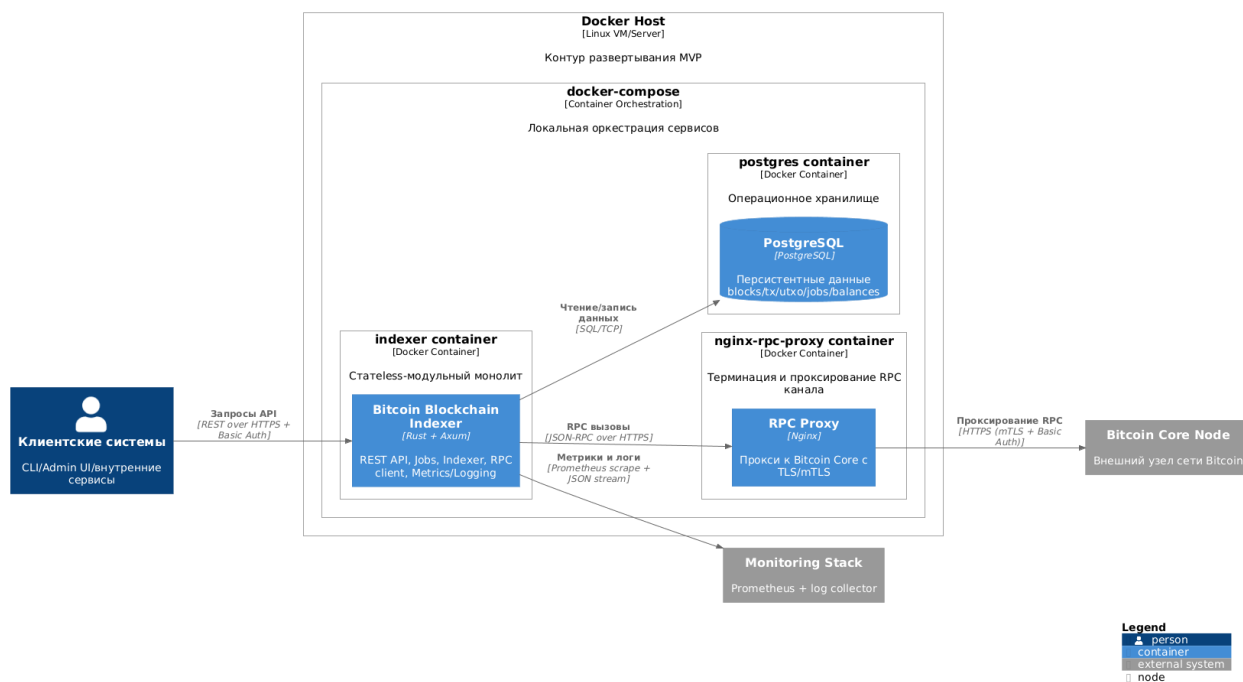


Рисунок А.1 – Контейнерное развёртывание системы

## ПРИЛОЖЕНИЕ Б

### Структура репозитория проекта

Таблица Б.1 фиксирует основные каталоги репозитория и их назначение. Она используется как справочный артефакт, связывающий текст пояснительной записки с фактической организацией исходного кода и документации.

Таблица Б.1 – Структура репозитория проекта

| Каталог      | Назначение                                        |
|--------------|---------------------------------------------------|
| src/         | серверные модули на Rust                          |
| admin-panel/ | административная панель на React/Vite             |
| cli/         | командный интерфейс на Python                     |
| config/      | конфигурация приложения                           |
| migrations/  | SQL-миграции PostgreSQL                           |
| doc/         | модульная документация по функциональным единицам |
| report/      | диаграммы и материалы лабораторного отчета        |
| diploma/     | шаблон и текст ВКР                                |

## ПРИЛОЖЕНИЕ В

### Фрагмент конфигурации индексатора

В приложении приведен сокращенный пример конфигурации индексатора. Полная конфигурация хранится в файле `config/indexer.yaml`; здесь оставлены только параметры, необходимые для понимания структуры настройки серверной части, RPC-подключения, индексатора и задания синхронизации.

Листинг В.1: Сокращенный пример конфигурации системы индексации

```
server:
  bind_host: "0.0.0.0"
  bind_port: 8080
  auth:
    basic:
      username: "admin"
      password_env: "INDEXER_API_PASSWORD"

rpc:
  node_id: "btc-testnet-1"
  url: "https://rpc.example.com"
  auth:
    basic:
      username: "rpcuser"
      password_env: "BITCOIN_RPC_PASSWORD"
  mtls:
    enabled: false

indexer:
  network: "testnet"
  reorg_depth: 12
  poll:
    tip_interval_ms: 5000
    mempool_interval_ms: 3000

jobs:
  - job_id: "full-sync"
    mode: "all_addresses"
    enabled: true
```